# Support d’installation réseaux

Ruddy Audigier Q1
Raphael Fouque
Pascal Hilt

**Lien google doc :**
https://docs.google.com/document/d/1uFZqvLo8PP_37fHCH9uhrYZ5aawKD-jp3zFn6DQ12yI/edit?usp=sharing

**Résumé :** Ce support guide l'étudiant dans la mise en place d'un réseau virtualisé complet sous Hyper-V. L'objectif est de configurer un routeur Linux assurant le NAT pour un réseau privé composé de clients et de serveurs (Web et Base de données), incluant le partage de fichiers inter-plateformes (SMB).

**Sommaire**

Introduction et Plan du réseau .. p.2

Configuration Hyper-V (Commutateurs) ...p.2

Création et Configuration du Routeur ... p.3

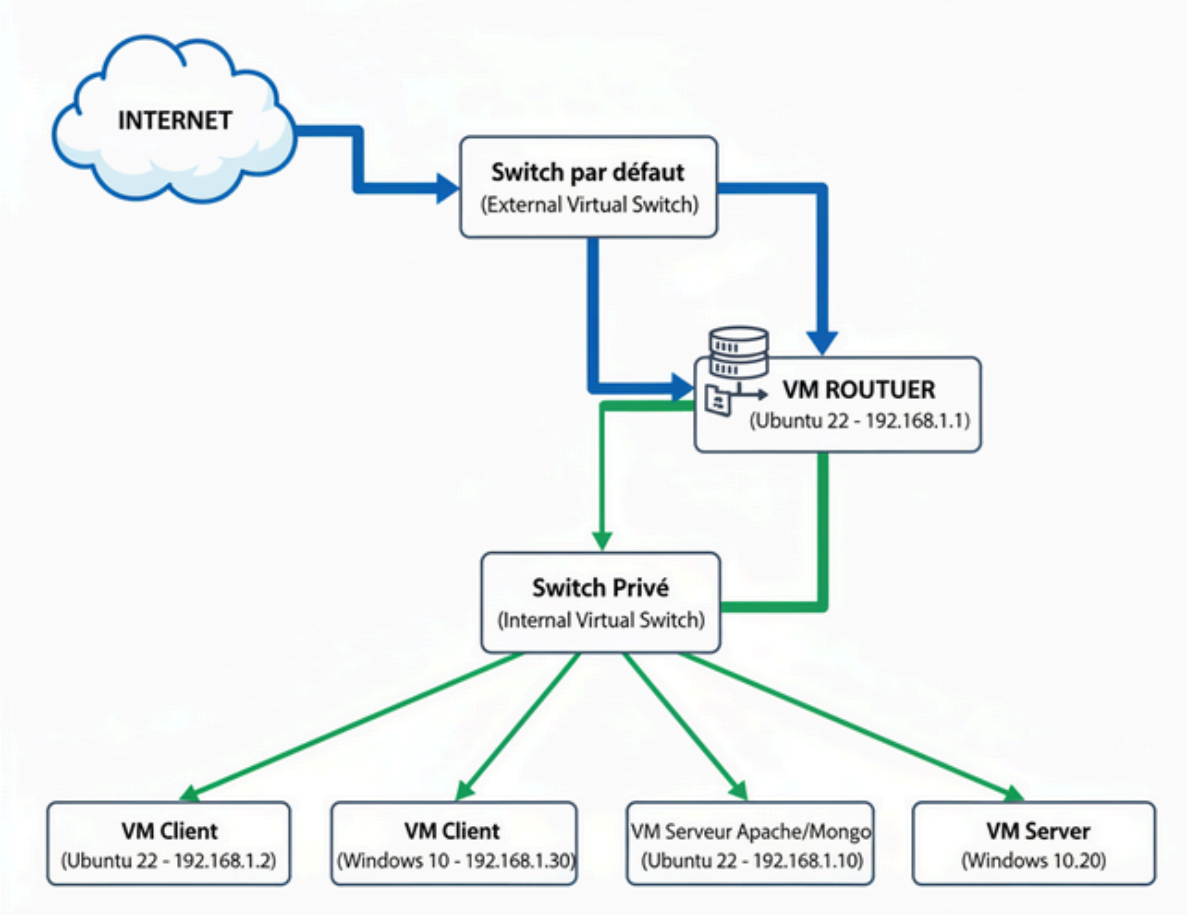
Installation des Clients (Linux & Win) ...p.6

Configuration des Serveurs (Apache/Mongo) p.8

Conclusion et Perspectives ....p.10

Tableau de conformité ..... p.11

**Introduction :** La virtualisation permet de simuler une infrastructure réelle sur une seule machine physique. Ici, nous utilisons Hyper-V pour isoler un réseau local ("Switch Privé") du réseau extérieur, tout en permettant l'accès internet via une passerelle Linux.




Le routeur est la seule machine possédant deux interfaces. Elle fait le pont entre le monde extérieur (eth0) et le réseau local (eth1).

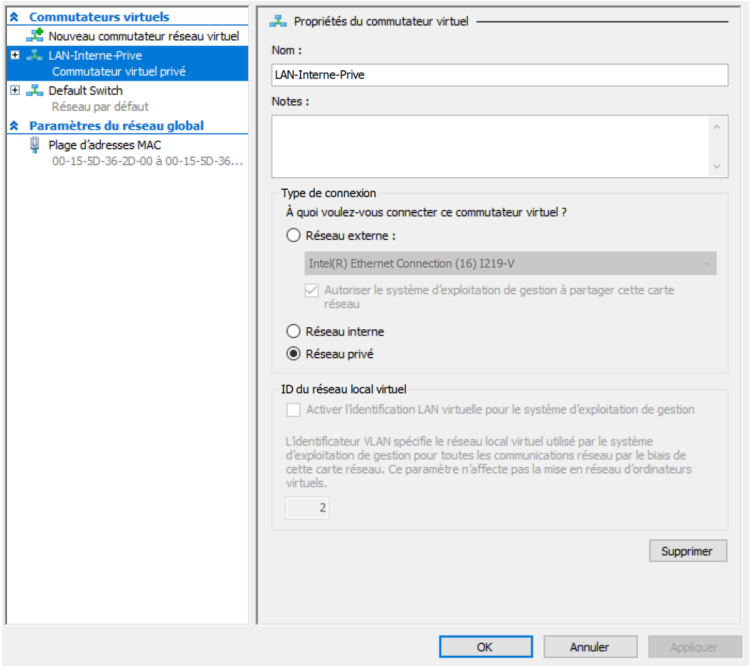
## Rappel de la configuration finale:

- VM Client sur Ubuntu 22 (1Go RAM / 2 coeurs / **ip -> 192.168.1.2**)
- VM Client sur Windows 10 (2Go RAM / 2 coeurs / **ip -> 192.168.1.30**)
- VM Routeur sur Ubuntu 22 (1Go RAM / 2 coeurs / **ip -> 192.168.1.1**)
- VM Serveur Apache + MongoDB sur Ubuntu 22 (2Go RAM / 2 coeurs / **ip -> 192.168.1.10**)
- VM Serveur quelconque sur Windows 10 (2Go RAM / 2 coeurs / **ip -> 192.168.1.20**)

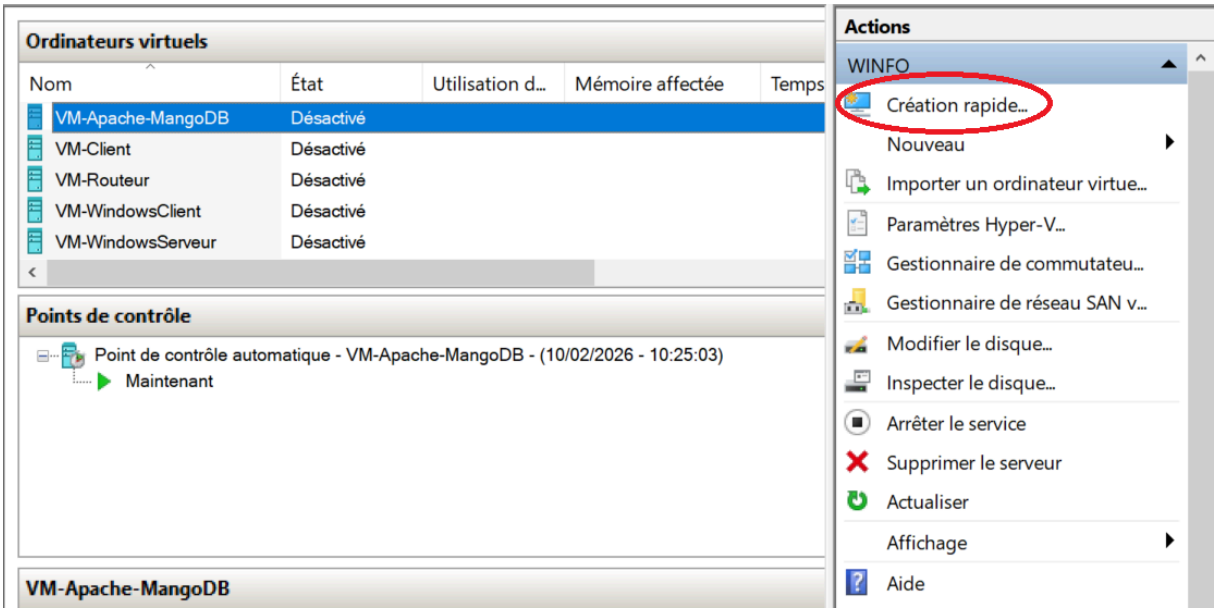
(Prévoyez 80 Go de stockage)

## Configurer les commutateurs sur Hyper-V:

- On garde le switch par défaut et on crée un switch privé



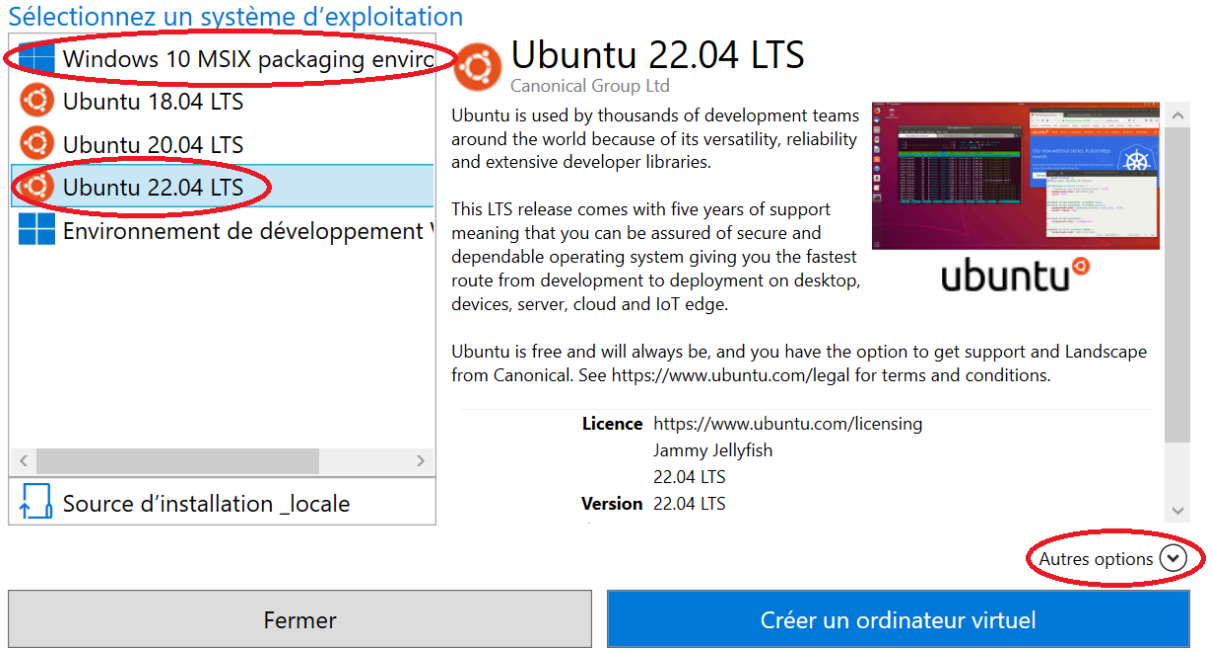

- Toutes les installations de VMs seront directement sur Hyper-V




- On utilisera seulement Windows 10 et Ubuntu 22

## On commencera par installer la VM Routeur (Ubuntu 22) :

- Choisir Ubuntu 22 puis cliquer sur “Autres options”
- On peut donner un nom à la VM et choisir la carte réseau qui sera le switch par défaut




- Avant de lancer la VM, il faut passer par les paramètres et effectuer quelques modifications

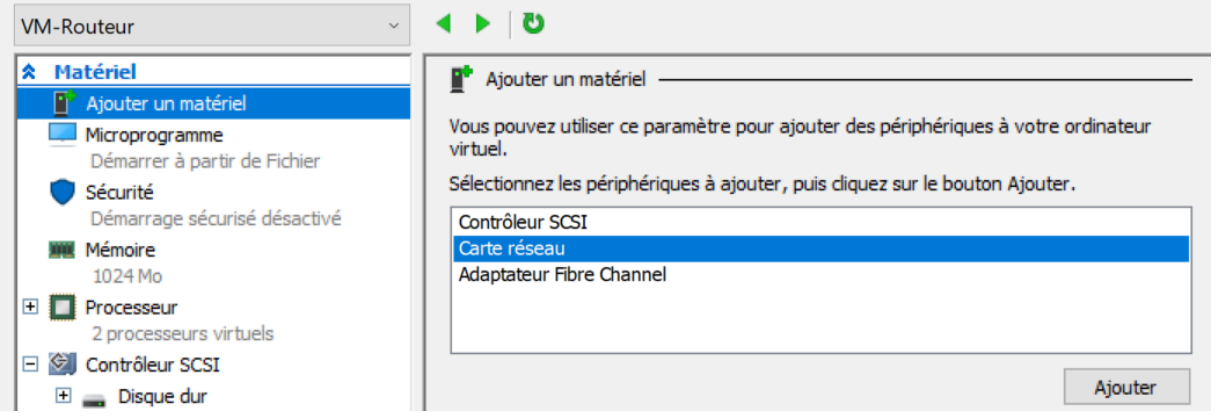
Création réussie de l'ordinateur virtuel

Ubuntu 22.04 LTS

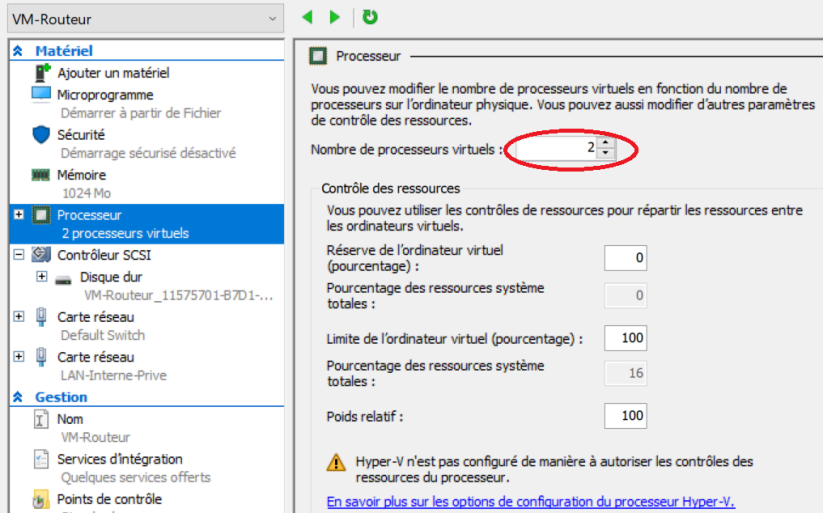
Se connecter

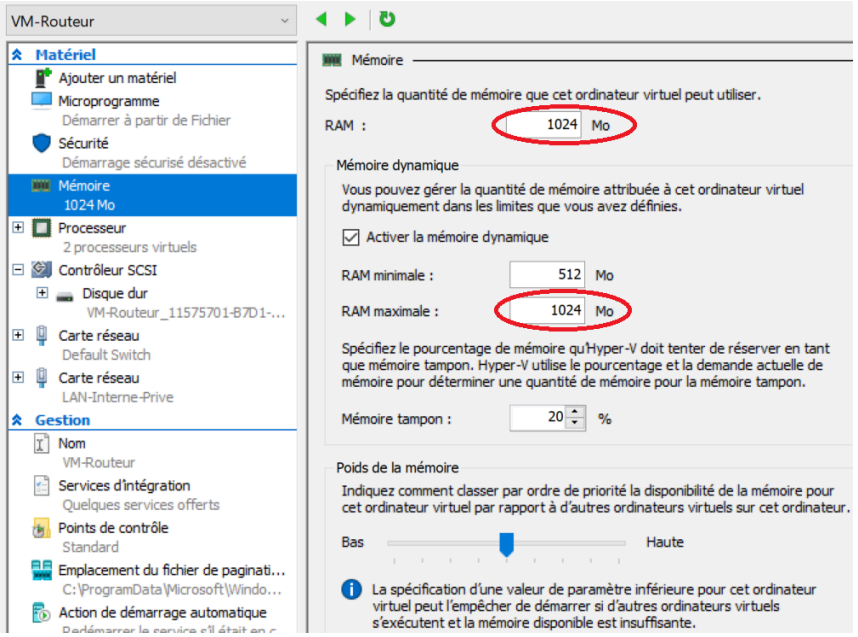
Modifier les paramètres...

- Il faudra rajouter l'autre carte réseau qui sera le switch privé déjà créée juste avant puis adapter les ressources à allouer (voir début du tuto)

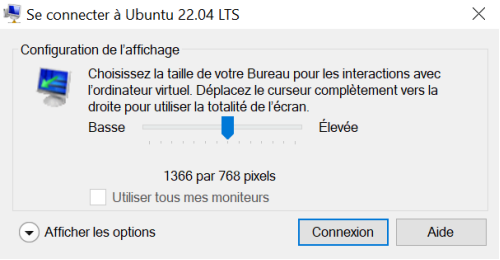



- Ensuite on va laisser 2 processeurs et 1Go de ram (1024 Mo) à la VM






- Vous pouvez maintenant lancer la VM et faire les configurations de base
- Choisissez la langue -> le clavier -> la région -> nom/pseudo/mot de passe (retenez le mot de passe)
- Une fois terminé, une fenêtre peut apparaître et vous demander de vous connecter mais vous pouvez simplement fermer la fenêtre




- Si vous avez appuyez sur “Connexion” quand même et que votre écran est bloqué sur un écran turquoise, appuyez simplement sur le logo “session simple” puis connectez vous

- Une fois connecté, on va devoir réaliser les configurations d’adresse ip. Cette VM est la seule à avoir le switch par défaut donc elle a accès à internet de base. Pour les autres VMs, il faudra donc les faire passer par cette VM pour qu’elles aient accès à internet.
- Ouvrez un terminal et écrivez : **sudo nano /etc/netplan/01-network-manager-all.yaml**
- Modifiez le fichier comme sur le screen
- Dans “eth0” et “addresses”, les adresses correspondent au DNS de la connexion de l’IUT, sans renseigner ce champ, la connexion à internet est impossible

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0: # Interface connectée à Internet
      dhcp4: true
      nameservers:
        addresses: [193.51.152.152, 193.51.152.153]
    eth1: # Interface du réseau privé
      dhcp4: no
      addresses:
        - 192.168.1.1/24
```

**Qu'est-ce que Netplan ?** Netplan n'est pas le service réseau lui-même, mais une "surcouche" (abstraction). Il lit ce fichier YAML pour générer la configuration réelle pour le moteur de rendu (ici systemd-networkd). Cela permet une configuration uniforme sur Linux, que vous soyez sur Hyper-V ou sur un serveur physique.

- Attention à ne pas inverser “eth0” et “eth1”, pour savoir lequel est le bon, tapez cette commande sur votre machine windows : **Get-VMNetworkAdapter -VMName "VM-client"**, vous aurez l’adresse MAC et vous pourrez ensuite la comparer avec celle de votre VM en faisant : **ip link show** (regardez comment se finit vos adresses MAC sur la VM)
- Sur mon screen, “eth0” correspond au switch par défaut et “eth1” au switch privé

```
PS C:\Users\cyr> Get-VMNetworkAdapter -VMName "VM-routeur"

Name         IsManagementOs VMName     SwitchName     MacAddress   Status IPAddresses
----         -------------- ------     ----------     ----------   ------ -----------
Carte réseau False          vm-routeur Acces-Internet 00155D010803 {Ok}   {}
Carte réseau False          vm-routeur LAN-Interne    00155D010804 {Ok}   {}
```

- Entrez maintenant ces commandes :
- **sudo systemctl start systemd-networkd**
- **sudo systemctl enable systemd-networkd**

- **sudo sysctl -w net.ipv4.ip_forward=1**
- **echo "net.ipv4.ip_forward=1" | sudo tee -a /etc/sysctl.conf**

- **sudo netplan apply**

- **sudo apt update**
- **sudo apt install -y iptables-persistent**
- **sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE**
- **sudo netfilter-persistent save**

**Pourquoi activer l'IP Forwarding ?** Par défaut, un noyau Linux refuse de transmettre des paquets d'une carte réseau à une autre pour des raisons de sécurité. En activant le net.ipv4.ip_forward, on transforme officiellement la VM en routeur capable de faire transiter le trafic des clients vers internet.

## Installation de la VM Client (Ubuntu 22) :

- Les étapes sont identiques à celles de l'installation de la VM Routeur jusqu'à la configuration de l'adresse ip
- Faites : "**sudo nano /etc/netplan/01-network-manager-all.yaml**", puis :

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: no
      addresses:
        - 192.168.1.2/24
      routes:
        - to: default
          via: 192.168.1.1 # Adresse de notre routeur
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

- Et pour finir :
- **sudo systemctl start systemd-networkd**
- **sudo systemctl enable systemd-networkd**
- **sudo netplan apply**

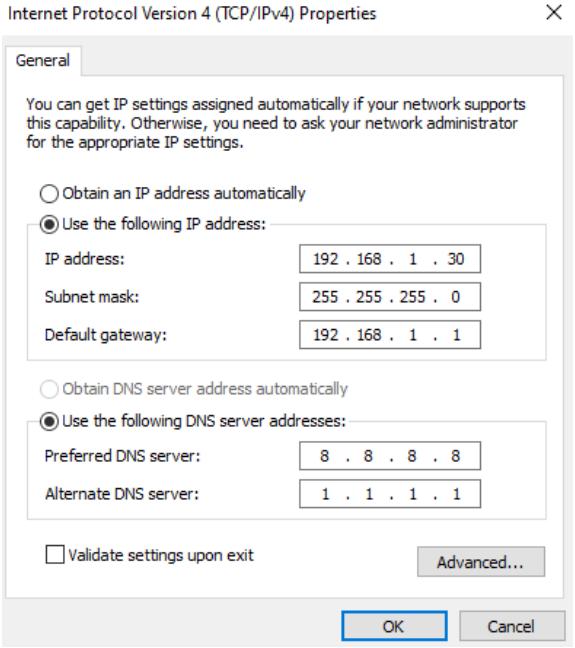
## Installation de la VM Apache MangoDB(Ubuntu 22) :

- Idem que pour la VM Client sauf que l'adresse ip est **192.168.1.10** (voir début du tuto)

## Installation de la VM Client Windows (Windows 10) :

- Créez une nouvelle VM Windows 10 (Création rapide / Nouveau) et donnez-lui un nom
- Dans les paramètres de la VM :
- Mettez 2 processeurs et 2 Go de RAM
- Vérifiez que la carte réseau est bien sur le switch privé
- Lancez la VM et faites l'installation de Windows normalement (langue / clavier / compte / mot de passe)
- Configuration de l'adresse IP (Client Windows) :
- Ouvrez : Panneau de configuration > Réseau et Internet > Centre Réseau et partage
- Cliquez sur : Modifier les paramètres de la carte
- Faites clic droit sur Ethernet > Propriétés

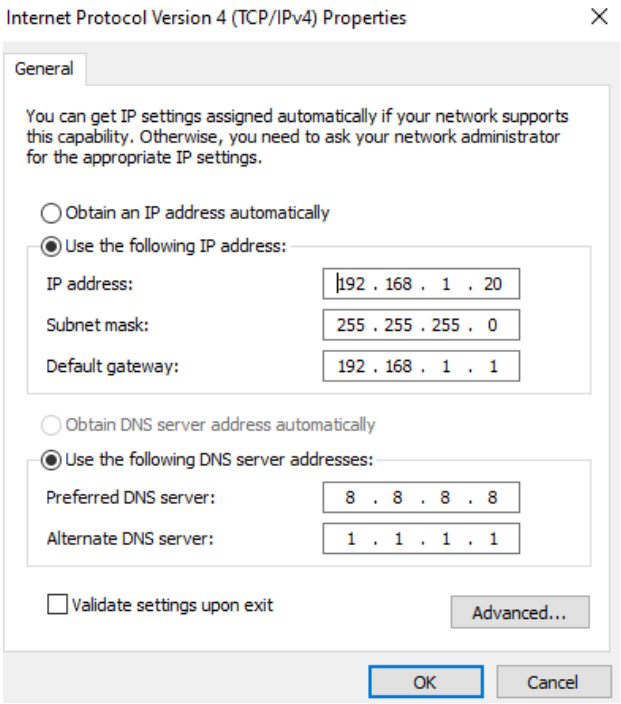
- Cliquez sur Protocole Internet version 4 (TCP/IPv4) > Propriétés




- Validez avec OK, puis refermez tout

## Installation de la VM Serveur (Windows 10) :

- Idem que pour la VM Client Windows (avec 192.168.1.20 en adresse ip)




- C'est fini pour les configurations de base, maintenant on va voir comment configurer le serveur Apache + MangoDB et un système de dossier partagé sur windows grâce au protocole SMB

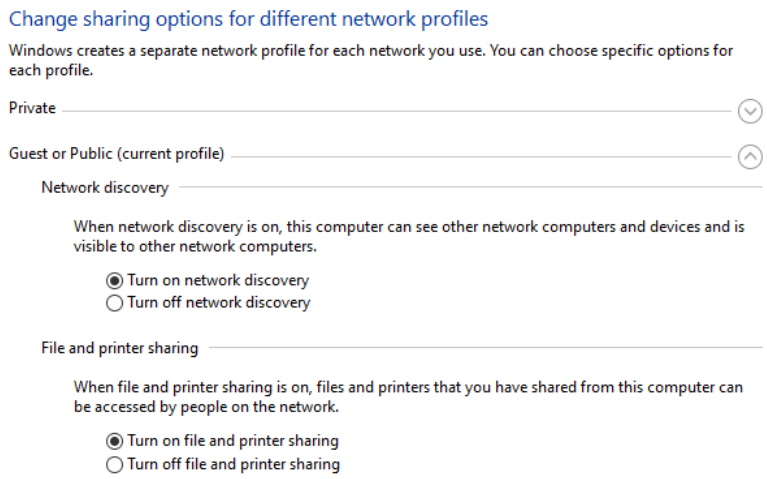
- Mais avant tout, vérifiez que chaque VM a accès à internet et vous pouvez ping 8.8.8.8 ainsi que toutes les autres VMs

## Configuration du dossier partagé sur VM Client Windows :

- Créez un dossier sur le bureau ou dans C:/ (ex : C:\Partage)
- Faites clic droit sur le dossier -> Propriétés
- Allez dans l'onglet Partage -> Partage avancé -> Cochez Partager ce dossier
- Cliquez sur Autorisations
- Cliquez sur Ajouter, tapez "Everyone", puis OK
- Cochez Autoriser sur : "Contrôle total"
- Cliquez sur Appliquer puis OK
- Allez dans l'onglet Sécurité (toujours dans Propriétés du dossier)
- Cliquez sur Modifier
- Ajoutez "Everyone" et donnez les droits à tous les utilisateurs et à "Everyone"
- Cliquez sur Appliquer puis OK

Vérifiez que le partage de fichiers est activé sur Windows :

- Paramètres > Réseau et Internet > Paramètres réseau avancés > Paramètres de partage avancés
- Activez : Découverte du réseau et Partage de fichiers et d'imprimantes




Le dossier partagé est maintenant fonctionnel

## Configuration du serveur Apache + MongoDB :

- Ouvrez un terminal sur la VM Serveur
- Mettez le système à jour : **sudo apt update sudo apt upgrade -y**
- Installez Apache : **sudo apt install apache2 -y**
- Vérifiez que le service est actif : **sudo systemctl status apache2**

- Si Apache n'est pas démarré :

**sudo systemctl start apache2**
**sudo systemctl enable apache2**

- Testez le fonctionnement en ouvrant un navigateur depuis une autre VM
- Tapez : **http://192.168.1.20**
- La page par défaut Apache doit s'afficher

- Installez MongoDB : **sudo apt install mongodb -y**
- Vérifiez que le service est actif : **sudo systemctl status mongodb**
- Si MongoDB n'est pas démarré : **sudo systemctl start mongodb sudo systemctl enable mongodb**
- Testez MongoDB : **mongo**
- Une fois dans le shell MongoDB : **show dbs**
- Pour quitter : **exit**

**<u>Mise en place d'une page Apache qui affiche des données MongoDB :</u>**

- L'idée est la suivante : MongoDB contient une petite collection (ex : des "employés") et la page web (en PHP) va se connecter à MongoDB, lire cette collection, puis afficher le résultat dans le navigateur.

Créer un exemple de base MongoDB :

- Ouvrez MongoDB sur la VM serveur Linux avec **mongo** (ou **mongosh**), puis créez une base et une collection avec un exemple de données :

```
use entreprise
db.employes.insertMany([
  { nom: "Dupont", poste: "Technicien", age: 25 },
  { nom: "Martin", poste: "Développeur", age: 21 }
])
exit
```

- Mettez à jour et installez PHP + Apache PHP + l'extension MongoDB :
- **sudo apt update**
- **sudo apt install -y php libapache2-mod-php php-cli php-mongodb**
- Redémarrez Apache pour prendre en compte PHP : **sudo systemctl restart apache2**
- Installer la librairie PHP MongoDB (via Composer) : sudo apt install -y composer
- Allez dans le dossier web et installez la librairie "mongodb/mongodb" :
- **cd /var/www/html**
- **sudo composer require mongodb/mongodb**
- Créez (ou modifiez) une page PHP dans /var/www/html, par exemple index.php :
- **sudo nano /var/www/html/index.php**
- Collez cet exemple (il se connecte à MongoDB en local, lit la collection "employes", et l'affiche) :

```php
<?php
require __DIR__ . '/vendor/autoload.php';

$client = new MongoDB\Client("mongodb://127.0.0.1:27017");
$collection = $client->entreprise->employes;

$employes = $collection->find();

echo "<h1>Employés (MongoDB)</h1>";
echo "<ul>";
foreach ($employes as $e) {
   $nom = htmlspecialchars($e["nom"]);
   $poste = htmlspecialchars($e["poste"]);
   $age = (int)$e["age"];
   echo "<li>$nom - $poste - $age ans</li>";
}
echo "</ul>";
?>
```

Le serveur Apache + MongoDB est maintenant bien configuré

## Comment accéder à chaque serveur ?

lancer le serveur Apache + SGBD ->

- sudo systemctl start/stop apache2 -> sudo systemctl start/stop mongod
- node server.js

accès au serveur Apache + SGBD -> entrez cette adresse dans votre navigateur : *http://192.168.1.10*

accès au dossier partagé depuis windows -> win + r -> \\192.168.1.20\Partage
accès au dossier partagé depuis ubuntu -> explorateur de fichier -> autre emplacement -> smb://192.168.1.20/Partage -> utilisateur -> “rentrez l’utilisateur et le mdp du compte de la VM Windows serveur”

## Conclusion

**1. Inventaire des équipements et services**

- [O] Passerelle Linux opérationnelle
- [X] Switch Interne 1 (Stations) et Switch Interne 2 (Serveurs)
- [O] 2 Stations (Ubuntu + Windows) sur Switch 1
- [O] 2 Serveurs (Ubuntu + Debian/Win) sur Switch 2
- [O] Serveur Web Apache/Nginx (sur Ubuntu)
- [O] Serveur de base de données SGBD (sur Ubuntu)

- [O] Service optionnel : dossier partagé (sur le 2ème serveur)

**2. Tests de connectivité et services**

- [O] Ping IP entre stations : **ok / échec**
- [O] Ping IP entre station et serveur : **ok / échec**
- [O] Ping IP entre station et Internet : **ok / échec**
- [?] Résolution DNS (nslookup) depuis une station : **ok / échec**
- [O] Accès HTTP au serveur Web depuis une station : **ok / échec**
- [?] Accès au serveur Web depuis l'Hôte (via l'IP publique de la passerelle) : **ok / échec**

**3. Robustesse**

- [O] Persistance après redémarrage (Windows Hôte / VM) : **ok / échec**

**Conclusion :** Ce projet nous a permis de comprendre le flux de données dans un réseau segmenté. La principale difficulté réside dans la configuration correcte des passerelles par défaut sur chaque machine.

**Réflexion sur Hyper-V :** Presque tout ce qui a été fait (Netplan, Apache, IP Forwarding) est indépendant d'Hyper-V. Seule la création des switchs virtuels est propre à l'interface Microsoft. On pourrait tout à fait automatiser cette installation via des scripts PowerShell ou des outils comme Ansible pour éviter l'interface graphique.

**Perspectives :** Pour améliorer ce réseau, on pourrait :

- Mettre en place un serveur DNS local (Bind9) pour ne plus utiliser les IP.
- Ajouter un Firewall plus restrictif sur le routeur pour isoler les serveurs des clients (zone DMZ).
- Automatiser le déploiement des VMs avec Vagrant ou Terraform.